Assignment 1 – PostgreSQL Basics (Homecare Database)

General Guidelines

- Use snake_case naming convention for all database objects.
- Primary keys must be UUID.
- Use PostgreSQL SQL syntax.
- Follow proper auditing practices.

1. Database Creation
    a. Create a PostgreSQL database named homecare_db.
2. Schema Creation
    a. Create a schema named hc inside the homecare_db database.
3. Users Table
    a. Create a users table with the following fields:
        i. user_id (UUID, Primary Key)
        ii. first_name
        iii. last_name
        iv. email
        v. password (encrypted)
        vi. created_by (user_id)
        vii. created_date
        viii. modified_by (user_id)
        ix. modified_date
        x. is_active
        xi. is_deleted
        xii. birth_date
        xiii. address
        xiv. mobile_number
4. Data Insertion
    a. Insert at least 10 user records with unique email addresses.
5. SQL Queries
    a. Write SQL queries for the following requirements:
        i. Fetch all users
        ii. Fetch only user name (first_name + last_name) and email
        iii. Fetch only inactive users
        iv. Users whose first_name starts with 'A' and last_name ends with 'i'

v. Users whose email contains '@example'
vi. Case-insensitive search for first_name starting with 'A'
vii. Users ordered by created_date (descending)
viii. Top 5 latest created users
ix. Second page of results (5 records per page)

Assignment 2: PostgreSQL Queries & Constraints

General Instructions

- Use PostgreSQL syntax only
- Follow snake_case naming convention
- Do not include query outputs
- Focus on correct SQL logic
- Database:traning_db
- Schema: tdb

Available Tables

- users
- roles
- service_types
- categories
- sub_categories

1. Aggregate Functions (COUNT, SUM, AVG, MIN, MAX)
    a. Count the total number of users
    b. Count the number of active users
    c. Count the number of users per role
    d. Find the minimum and maximum birth_date of users
    e. Calculate the average age of users
    f. Count total categories per service type
    g. Count total sub-categories per category
2. GROUP BY & HAVING
    a. Fetch the number of users grouped by role
    b. Fetch roles having more than 2 users
    c. Fetch service types having more than 3 categories
    d. Fetch categories having at least 2 sub-categories
    e. Fetch users grouped by birth year
    f. Fetch birth years having more than 5 users
3. Joins
    a. INNER JOIN:
        i. Fetch categories with service type names
    b. LEFT JOIN:
        i. Fetch all service types including those without categories

    c. RIGHT JOIN:
        i. Fetch all categories even if they have no sub-categories
4. EXISTS
    a. Fetch roles that have at least one user
    b. Fetch service types that have categories
    c. Fetch categories that have sub-categories
    d. Fetch users whose role exists
5. Common Table Expression (CTE)
    a. Use a CTE to count the number of categories per service type
6. Constraints
    a. UNIQUE Constraint:
        i. Ensure users.email is unique
    b. FOREIGN KEY Constraints:
        i. users → roles
        ii. categories → service_types
        iii. sub_categories → categories
    c. CHECK Constraints:
        i. Mobile number must have a valid length
        ii. birth_date must be a past date
7. Indexes
    a. Create an index on users.email
8. Date & Time Handling
    a. Fetch users created today
    b. Fetch users created in the last 30 days
    c. Extract year from users.birth_date
    d. Calculate age of each user
    e. Group users by birth year
9. Window Functions (ROW_NUMBER, RANK, LAG, LEAD)
    a. Assign row numbers to users within each role based on created_date
    b. Rank roles based on number of users
    c. Use LAG to fetch previous user creation date per role
    d. Use LEAD to fetch next user creation date per role
    e. Fetch the second oldest user per role

Assignment 3 – Advanced PostgreSQL

1. Views & Materialized Views
    a. Create a **VIEW** that displays:
        i. user_id
        ii. full_name (first_name + last_name)
        iii. email
        iv. role_name
    b. Create a **VIEW** that shows only **active users** created in the last 30 days.
    c. Create a **MATERIALIZED VIEW** that stores:
        i. role_name
        ii. total_users per role
    d. Write a query to **refresh** the materialized view.
2. Functions
    a. Create a **SQL function** that:
        i. Accepts a role_id as input
        ii. Returns the total number of users for that role
    b. Create a **PL/pgSQL function** that:
        i. Accepts a user_id
        ii. Returns the user's full name
    c. Create a function that:
        i. Accepts birth_date
        ii. Returns calculated age
    d. Create a function that:
        i. Returns all users created **today**
    e. Demonstrate usage of:
        i. IN parameters
        ii. OUT parameters
        iii. RETURN TABLE
3. Stored Procedures
    a. Create a **stored procedure** to:
        i. Insert a new user

    ii. Validate email uniqueness
    iii. Set created_date automatically
  b. Create a stored procedure that:
    i. Soft deletes a user (is_deleted = true)
  c. Create a stored procedure that:
    i. Updates user role
  d. Logs old role and new role into an audit table
4. Triggers
  a. Create a trigger that:
    i. Automatically updates modified_date on UPDATE of users table
  b. Create a trigger that:
    i. Prevents deletion of users
    ii. Raises an exception if DELETE is attempted
  c. Create an **AFTER INSERT** trigger to:
    i. Log user creation into an audit table
5. Cursors
  a. Write a **PL/pgSQL block** using a cursor to:
    i. Loop through all users
    ii. Print user_id and email using RAISE NOTICE
6. Jobs / Scheduling
  a. Write a job to:
    i. Refresh a materialized view every hour

Assignment 4 - PostgreSQL Advanced Level

1. Transactions & ACID
    a. Write a transaction that:
        i. Inserts a new user
        ii. Updates user status
        iii. Commits only if both operations succeed
    b. Write a transaction that:
        i. Inserts data into two tables
    c. Rolls back if any statement fails
2. Security Basics
    a. Create database roles for:
        i. db_admin
        ii. app_owner
        iii. app_user
        iv. app_readonly
    b. Grant appropriate privileges using:
        i. GRANT
    c. Revoke privileges using:
        i. REVOKE
    d. Demonstrate:
        i. Schema-level security
        ii. Table-level security
    e. Revoke all default privileges from:
        i. PUBLIC role on database
        ii. PUBLIC role on schema
    f. Grant appropriate privileges on the schema to:
        i. app_user
        ii. app_readonly
    g. Table & Object Privileges
        i. Grant permissions to app_user to:
            1. SELECT
            2. INSERT

3. UPDATE
4. DELETE

   on all tables in schema hc.

ii. Grant permissions to app_readonly to:
    1. SELECT only

       on all tables in schema hc.

iii. Prevent app_user from:
    1. Dropping tables
    2. Truncating tables